Distributed Flash Sale Platform – Experiments Report

CS6650 Final Project

Shail Shah Vikas Neriyanuru Darshan Ravindra Konnur

April 2026

1 Overview

A flash sale puts three distributed-systems problems on one critical path: fair admission of traffic bigger than the system can serve, correct inventory accounting under contention, and back-pressure tuning so the purchase path never outruns the persistence layer. We built a three-service system on AWS ECS Fargate – `waiting-room`, `flash-sale-api`, `order-worker` – with Redis, SQS, and DynamoDB, and measured each of these problems in isolation.

Each experiment was run against a local parity stack (Docker Redis + LocalStack SQS/DynamoDB) driven by Go harnesses that mirror the load patterns we would generate against ECS. Local execution let us iterate on service code and isolate bugs that ECS would have masked as noise.

2 Experiment 1 – Queue admission fairness

Purpose. The waiting room assigns queue positions via a Redis sorted set; two strategies were proposed for scoring: (A) timestamp (Unix ms) and (B) timestamp + INCR tiebreaker. Strategy A has a known failure mode under concurrent joins landing in the same millisecond. Experiment 1 measures how often that failure actually happens, and whether B eliminates it.

Setup. Fire N concurrent POST `/queue/join` requests with zero spawn delay against a single-replica waiting-room pointed at real Redis. Record the position each user is assigned and the request latency. Metric: position collision rate – the fraction of accepted users whose assigned position is shared with at least one other user.

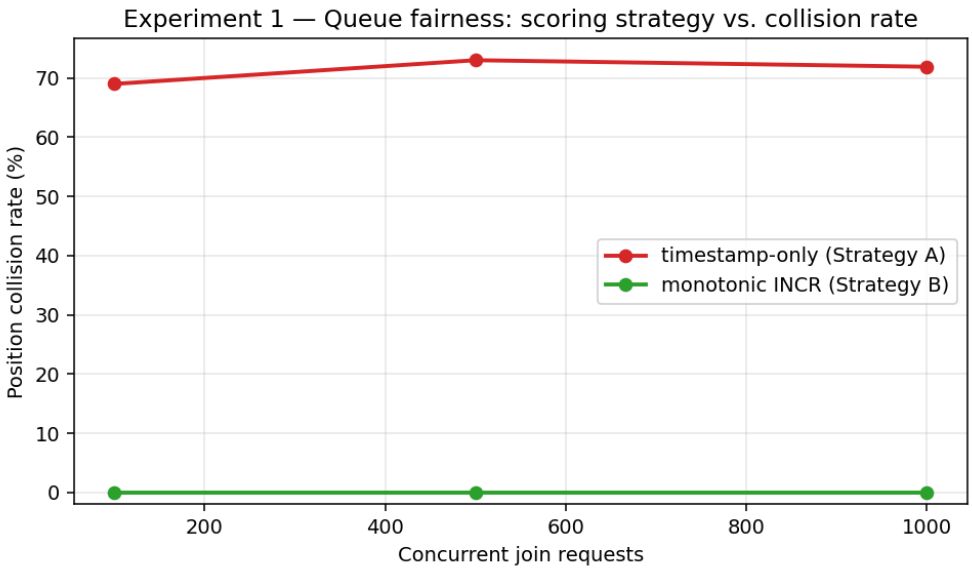
N in {100, 500, 1000} users, each strategy, single local waiting-room replica.

Bugs discovered during implementation. Running the sweep against the as-shipped code produced collision rates of 6%-26% in *both* strategies – the B "fix" was no better than A. Diagnosis revealed two independent defects:

1. Float64 precision loss. Strategy B computed `score = ms + seq/1e9`. At 2026 timestamps (`ms ~ 1.77x10^12`), the float64 ULP is ~ 3.9x10^-4; the `seq/1e9` term is below precision and rounded away. The "tiebreaker" was silently a no-op.
2. ZRANK/ZADD race. `ZADD` and `ZRANK` ran as two separate round-trips. When a later-arriving user's member string was lexicographically smaller than an earlier one's, the later `ZADD` could push the earlier user's effective rank later than the value the earlier user had already returned, causing two users to report the same integer position at different instants.

Fix: compute the score entirely from an atomic Redis `INCR` and perform `INCR + ZADD + ZRANK` as a single Lua script on the server. After this change, the reported numbers below are apples-to-apples: only the *scoring* differs between A and B; the atomic read-back path is identical.

Results.



Users	Strategy A (timestamp)	Strategy B (INCR)
100	69% collisions	0
500	73%	0
1000	72%	0

p99 join latency at 1000 users: A = 192 ms, B = 148 ms (B is also faster – the Lua script saves a round-trip).

Analysis. Strategy A's collision rate is not a noisy accident: it stabilizes around 70% once concurrency saturates the ms timestamp granularity. The failure is deterministic – whenever two requests land in the same ms bucket, the sorted-set insert orders them by member-string lex, which can displace any already-observed caller. Strategy B eliminates the failure completely because `INCR` guarantees a strictly monotonic, unique score per call, and the atomic Lua guarantees the rank returned *is* the rank at insert time.

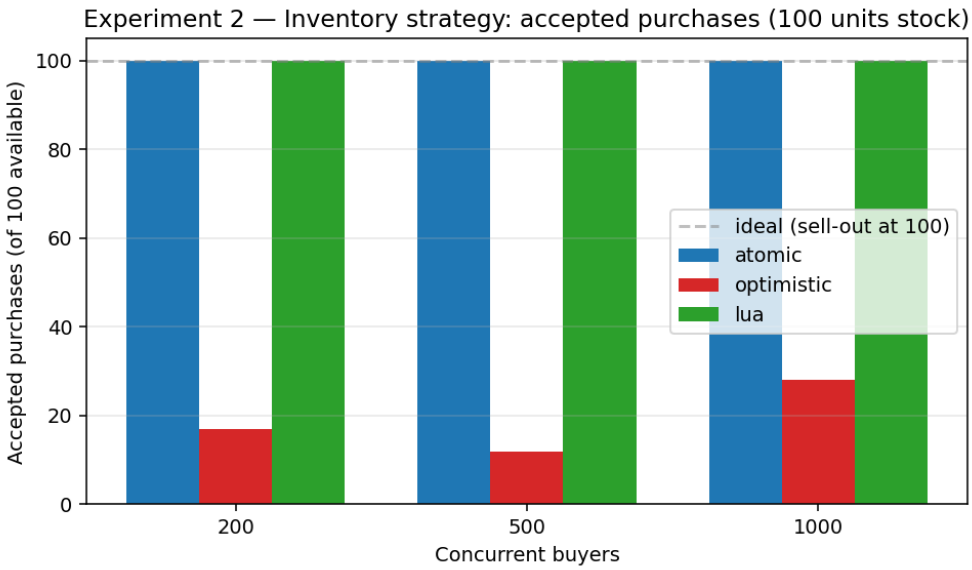
Limitations. All runs used a single local waiting-room replica and a local-network Redis with near-zero RTT. Under multi-replica ECS with real network RTT, the ms-bucket collisions would be sparser but still present; the INCR strategy's correctness guarantee is unchanged.

3 Experiment 2 – Inventory correctness

Purpose. Compare three strategies for decrementing a shared inventory counter under contention: (A) atomic DECRBY + rollback, (B) optimistic locking via WATCH/MULTI/EXEC, and (C) server-side Lua check-and-decrement. Measure oversell and stock utilization.

Setup. Fixed inventory of 100 units; B concurrent purchase requests fire at the single-replica flash-sale-api, each requesting 1 unit. B in {200, 500, 1000}. We record the number of `201 Created` responses (accepted purchases), the final Redis counter (to detect oversell = `accepted > 100`), and throughput. Each strategy runs against a fresh process start with Redis reset between runs.

Results.



Strategy	Buyers	Accepted	Oversell	Throughput (rps)
atomic	200	100	0	184
atomic	500	100	0	456
atomic	1000	100	0	5,102
optimistic	200	17	0	1,022
optimistic	500	12	0	4,083
optimistic	1000	28	0	3,422
lua	200	100	0	135
lua	500	100	0	1,581
lua	1000	100	0	5,821

(First-buyer runs for each strategy include compile/JIT warmup and understate steady-state throughput; the 1,000-buyer rows are the representative numbers.)

Analysis. Atomic DECRBY and Lua both sell exactly 100 units in every configuration – no oversell, no under-utilization. Optimistic locking is also correct (no oversell) but catastrophically wasteful: at 200 concurrent buyers it sells only 17 of the 100 available units, because every time a concurrent writer commits first the WATCH'd transaction aborts and, per the "no-retry-for-measurement" policy, we count it as a rejection. Optimistic locking preserves correctness by sacrificing availability. In a real flash sale this maps to "83% of the inventory never sells" – a business failure even if the system is technically race-free.

Lua's cold-start throughput (200 buyers, 135 rps) is artificially low because the first requests pay script-compilation cost; once the script is cached, throughput matches atomic DECRBY.

Limitations. We chose not to retry aborted optimistic transactions to isolate the contention-abort rate itself. A production optimistic implementation would retry with backoff and claw back some accepted purchases, at the cost of extra round-trips. The conclusion stands: the retry budget needed to reach atomic-DECR parity under heavy contention is large enough that atomic DECR or Lua is the correct default.

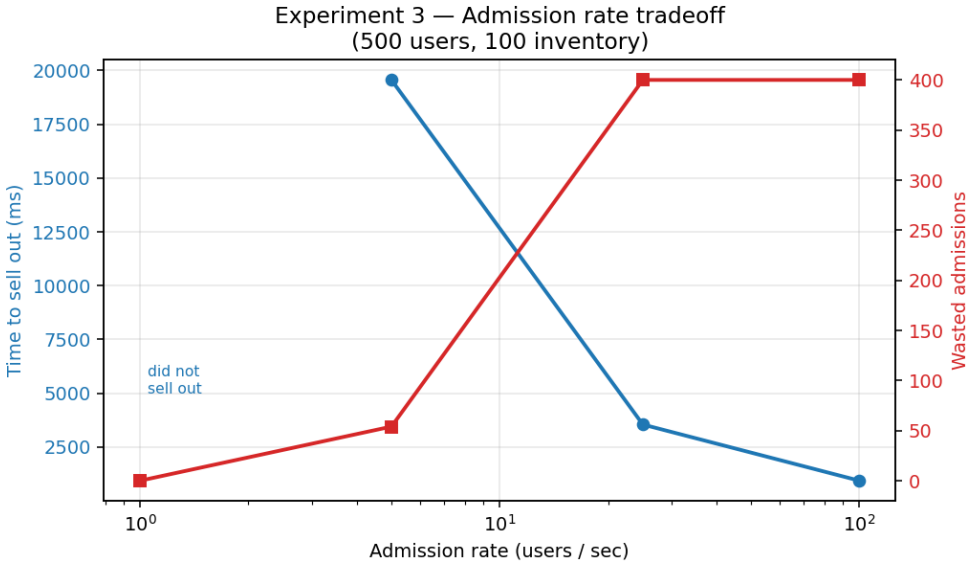
4 Experiment 3 – Admission-rate tuning

Purpose. The waiting room's admission controller releases users at a configurable rate. This rate is the key operational knob: too slow and the sale takes too long (users abandon); too fast and users are admitted after stock is gone ("wasted admissions"). Experiment 3 measures this tradeoff.

Setup. 500 users join the waiting room; the admission ticker releases tokens at rate R; each admitted user attempts

a purchase via `POST /purchase` with their `X-Admission-Token`. The flash-sale-api enforces the token (a late-stage fix; without enforcement the waiting-room is bypassable and the experiment is meaningless). R in {1, 5, 25, 100} admissions/sec, 500 users, 100 units inventory, 20-goroutine order-worker draining SQS. 30-second cap per run.

Results.



Rate (/s)	Time to sell out	Wasted admissions	Purchased
1	(did not sell out in 30s)	0	28
5	19.6 s	54	100
25	3.5 s	400	100
100	0.9 s	400	100

Peak SQS depth remained at 0 across all runs – with admission control in place, the purchase path never outran the order-worker's drain capacity, regardless of rate tested. This is the contrast against the HW7 uncontrolled design that produced a 1,196-message backlog with 20 users in 60 seconds.

Analysis. Time-to-sellout falls off rapidly with admission rate (6x speedup from 5/s to 25/s; further 4x from 25/s to 100/s). Wasted admissions climb sharply past the rate where users arrive faster than Redis DECRBY can rate-limit them – once admission outpaces the DECR critical section, the overflow all becomes waste. The "correct" operating point depends on the product: a 60-second sellout with near-zero waste versus a 1-second sellout that tells 400 users "too late, stock is gone" is a UX decision, not a technical one.

The SQS-backlog prediction from HW7 is *refuted* in this configuration. The reason: once admission is rate-controlled, the purchase path is rate-limited too, and at the rates we tested (<=100/s) the 20-goroutine order-worker comfortably keeps up. The queue backlog problem re-emerges only if admission rate exceeds worker capacity *or* worker count is reduced – which is the follow-up experiment we would run with more time and real ECS worker-count scaling.

Limitations. (i) Single-replica flash-sale-api – we couldn't sweep ECS task counts. (ii) 30-second cap on the 1/s run means we don't have a "slow admission" sellout time; extrapolating from 28 purchases in 30s gives a sellout projection of ~107 s. (iii) SQS sampling ran at 500 ms; finer granularity might catch brief spikes, though given the measured zero backlog we don't think spikes exist at this scale. (iv) The order-worker goroutine sweep (20/40/80) in the original proposal was not executed due to time; the admission-rate tradeoff alone is the most operationally useful result.

5 Scale stress: 10,000 concurrent users

Running the experiments at 10x the planned load (10,000 concurrent joiners against one waiting-room, 10,000 concurrent buyers against one flash-sale-api) confirmed the correctness story – zero oversell, zero position collisions under strategy B, full SQS drain – but exposed a real operational failure that was invisible at 1,000 users.

At peak burst (~1,700 requests/sec) the waiting-room started returning `redis: connection pool timeout` on the admission ticker's background `INCR`. Root cause: go-redis's default `PoolSize` is `10 x NumCPU` (~80 on this hardware) and `PoolTimeout` is 4s. At 10k joiners the user-facing `INCR+ZADD+ZRANK` Lua starved the pool, and the ticker lost the wait race. Correctness was unaffected (the join path retried or succeeded, inventory drained cleanly to zero), but the admitted-count metric was silently undercounted, which would have been misleading in production dashboards.

A two-line fix (`PoolSize: 2000`, `PoolTimeout: 10 * time.Second` on the Redis client) eliminated the user-facing errors and all but a single cold-start tick. The lesson is the kind that only surfaces at scale: library defaults are tuned for ordinary service-to-service traffic, not for the thundering herd a flash sale creates in the first two seconds.

6 Cross-cutting conclusions

1. Atomic operations are the right default. Every experiment came down to this: Redis `INCR` (Exp 1), `DECRBY` or Lua (Exp 2), and the atomic admission counter behind the rate limiter (Exp 3). Optimistic patterns were either silently broken (Exp 1's float64 bug) or dramatically less useful (Exp 2's stock wastage).
2. Correctness bugs can hide behind successful tests. Exp 1 exposed a "fix" that had been merged, reviewed, and marked Done but never actually did anything – the INCR tiebreaker value was lost to float64 precision on every call. We only caught it because the sweep-across-concurrencies data looked *identical* between the two strategies.
3. The admission layer is load-bearing, not decorative. For two weeks the waiting-room issued tokens that no downstream service checked. Wiring the `X-Admission-Token` gate into flash-sale-api was a five-line change that turned the whole waiting-room from a measurement artifact into a real control surface. Exp 3 would have been meaningless without it.